ARCHITECTURE & DESIGN

DOCUMENT 3 OF 5

SDD

Infralytix

Software Design Document

Layered architecture, multi-agent orchestration design, database schema & ER diagram, REST API contracts, key workflow sequences, and the design decisions behind them.

Layered Architecture | Multi-Agent Orchestrator | ER Diagram | REST API Contracts

Realizes: Infralytix SRS v1.0

Version 1.0 · July 2026

Table of Contents

Software Design Document

1 Introduction & Design Goals

Why the architecture looks the way it does

This Software Design Document (SDD) translates the requirements in the SRS (Document 2 of 5) into a concrete architecture: how the system is layered, how the multi-agent AI layer is orchestrated, how data is modeled, and how the REST API is shaped. It is written for the engineer implementing Infralytix and for anyone auditing the design.

Design Goal	How the architecture supports it
Modularity	Each Intelligence Module maps 1:1 to a backend service package and a specialized Agent — independently testable
Extensibility	New Intelligence Modules are added as new Agents registered with the Orchestrator, without touching existing agents
Graceful degradation	Agent failures return partial results with a clear error rather than failing the whole request
Portfolio clarity	Layering and naming are intentionally explicit (not over-abstracted) so the design reads clearly in a code review or demo

2 System Architecture

Layered architecture & folder structure

Infralytix follows a six-layer architecture. Requests flow top to bottom; each layer only calls the layer directly beneath it, keeping responsibilities cleanly separated.

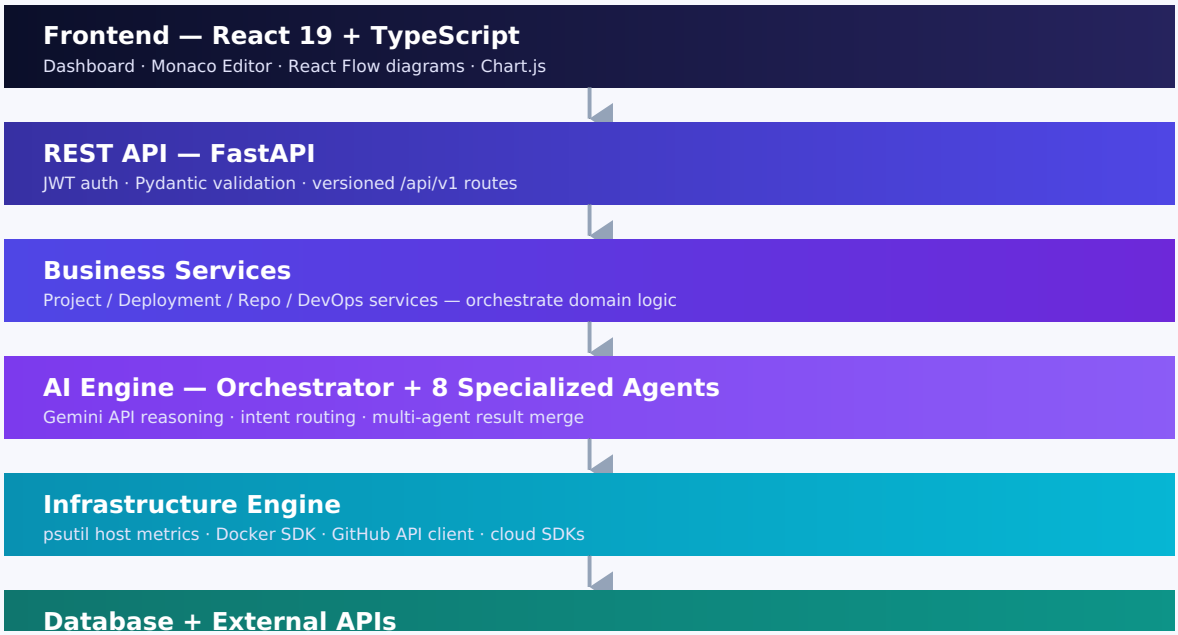

Fig. 1 — Six-layer architecture. Each layer depends only on the layer beneath it.

Folder Structure

```
Infralytix/
├── frontend/                React 19 + TS SPA
├── backend/
│   ├── api/                 FastAPI routers (one per Intelligence Module)
│   ├── services/             Business Services layer
│   ├── ai_agents/            Orchestrator + 8 specialized Agents
│   ├── deployment/           Dockerfile / Compose / CI generation logic
│   ├── monitoring/           psutil + Docker SDK collectors
│   ├── security/             Secret scanning, config auditing
│   ├── github/               GitHub OAuth + REST client
│   ├── reports/              Report Intelligence Agent output assembly
│   ├── database/             SQLAlchemy models + Alembic migrations
│   ├── auth/                 JWT issuance, RBAC middleware
│   ├── models/               Pydantic schemas
│   └── main.py               FastAPI app entrypoint
├── uploads/                 Temporary repo/log uploads
├── generated/                Generated Dockerfiles, workflows, diagrams
├── logs/
├── docs/                     This documentation suite
├── docker/                   Dockerfile + docker-compose.yml
├── tests/
└── README.md
```

3 Multi-Agent Orchestration Design

The Orchestrator, eight Agents, and routing logic

Every request that needs AI reasoning — whether a direct module action or a free-form AI Engineer question — passes through the **Orchestrator**. The Orchestrator classifies intent, dispatches to one or more Agents, and merges their output via the Report Intelligence Agent before returning a single response to the API layer.

Fig. 2 — Orchestrator fans a request out to the relevant Agent(s); the Report Intelligence Agent merges their output into one response.

Routing Logic (Simplified)

1. **Intent classification** — the Orchestrator sends the request (plus lightweight context: project id, recent history) to Gemini with a routing-only prompt, returning a list of relevant Agent names.
2. **Fan-out** — each selected Agent runs independently against its own domain data (repo contents, logs, metrics, etc.), in parallel where possible.
3. **Fan-in** — the Report Intelligence Agent receives all Agent outputs and produces one coherent, de-duplicated response, preserving structured data (tables, code) for the frontend to render natively.

DESIGN DECISION

Phase 1–2 ship each Agent behind its own direct API endpoint (no Orchestrator yet) so each module is independently demoable early. The Orchestrator is introduced in Phase 4 once all eight Agents exist and are individually proven — see Section 7.

4 Database Design

ER diagram and data dictionary

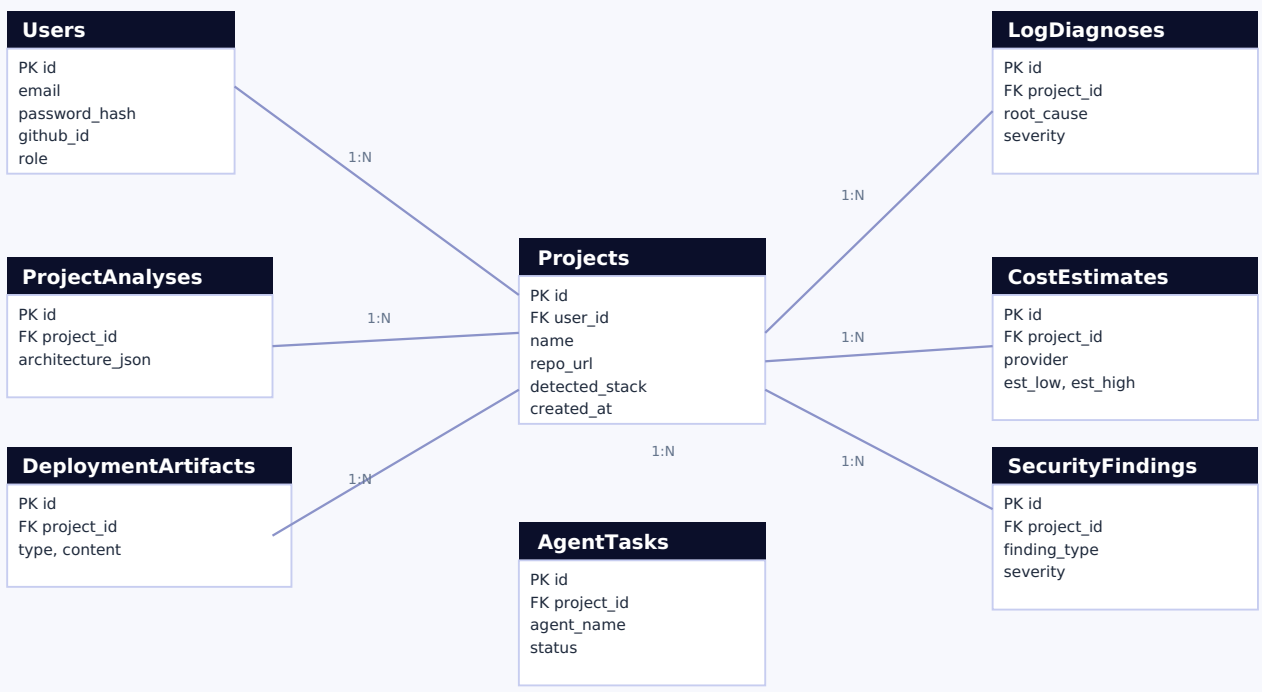

Fig. 3 — Projects is the central entity; every Intelligence Module writes its output to its own table keyed by project_id.

Data Dictionary (Key Tables)

Table	Key Columns	Purpose
users	id, email, role	Auth identity; role drives RBAC (Owner/Collaborator/Viewer)
projects	id, user_id, repo_url	Central hub — every module's output references project_id
project_analyses	id, project_id	Project Intelligence output — stack, deps, structure (versioned)
deployment_artifacts	id, project_id, type	Generated Dockerfile / Compose / workflow / env template content
log_diagnoses	id, project_id, severity	Log Intelligence root-cause + fix history
infra_metrics	id, project_id, captured_at	Time-series CPU/memory/disk/container samples
cost_estimates	id, project_id, provider	Cloud Intelligence estimate ranges + assumptions
security_findings	id, project_id, severity	Redacted secret/config findings — never raw secret values
repo_audits	id, project_id	Repository Intelligence — dead code, duplication, large files
architecture_diagrams	id, project_id, diagram_type	Architecture Intelligence generated diagram definitions (JSON for React Flow)
agent_tasks	id, project_id, agent_name, status	Orchestrator execution log — one row per Agent invocation

5 API Design

REST endpoints — one group per Intelligence Module, all under `/api/v1`

Method	Endpoint	Description
POST	/auth/signup, /auth/login	Register / authenticate; returns JWT
POST	/auth/github/callback	GitHub OAuth login callback
POST	/projects	Create project from upload or GitHub URL
GET	/projects / /projects/:id	List / fetch project(s)
POST	/projects/:id/analyze	Trigger Project Intelligence Agent
POST	/projects/:id/deployment/generate	Trigger Deployment Intelligence Agent
GET	/projects/:id/infra/metrics	Poll Infrastructure Intelligence live metrics
POST	/logs/diagnose	Submit a log/traceback to Log Intelligence Agent
POST	/projects/:id/cost/estimate	Trigger Cloud Intelligence Agent
POST	/projects/:id/security/scan	Trigger Security Intelligence Agent
POST	/projects/:id/repo/audit	Trigger Repository Intelligence Agent
POST	/projects/:id/devops/generate	Trigger DevOps Intelligence Agent
GET	/projects/:id/architecture/diagram	Fetch/generate Architecture Intelligence diagram JSON
POST	/ai/ask	Free-form AI Engineer query → Orchestrator (Phase 4)

CONTRACT CONVENTION

Every module endpoint returns a common envelope: `{ data, agent, generated_at, warnings[] }` so the frontend can render any module's output with one shared result-panel component.

6 Key Workflow Sequences

Step-by-step system behavior for core flows

Workflow A — Diagnose a Failed Deployment

Step	System Behavior
1	User pastes a stack trace / log into the Log Intelligence panel
2	FastAPI `/logs/diagnose` validates payload, forwards to Log Intelligence Agent
3	Agent constructs a Gemini prompt with the log plus known project context (stack, recent deploys)
4	Gemini returns structured root cause, severity, and fix suggestion
5	If the fix implies a config change, the Deployment Intelligence Agent is invoked to regenerate the affected file
6	Response envelope returned to frontend; result rendered with a "view diff" action on any regenerated file

Workflow B — Generate Deployment Configuration From Scratch

Step	System Behavior
1	User uploads a repo with no existing Docker/CI configuration
2	Project Intelligence Agent detects stack, entry point, and dependency manifest
3	Deployment Intelligence Agent generates Dockerfile, docker-compose.yml, and a GitHub Actions workflow
4	Generated files are written to `generated/` and shown in the Monaco Editor for review
5	Report Intelligence Agent assembles a short deployment guide summarizing what was generated and why

Workflow C — Multi-Cloud Cost Estimate

Step	System Behavior
1	User requests a cost estimate for the current or a described workload
2	Cloud Intelligence Agent maps the workload to comparable instance classes on AWS, Azure, and GCP
3	Provider pricing APIs are queried server-side; a ranged estimate (low/high) is computed per provider
4	Agent generates at least one concrete optimization suggestion (e.g. right-sizing, reserved instances)
5	Result rendered as a comparative table across the three providers with assumptions listed inline

7 Design Decisions & Trade-offs

The calls we made, and why

Decision	Alternative Considered	Why This Choice
Ship modules as direct endpoints first, Orchestrator later (Phase 4)	Build the Orchestrator first, modules behind it from day one	Faster path to a demoable product; de-risks each Agent before adding routing complexity
Synchronous agent calls in v1.0	Async task queue (Celery/Redis) from day one	Celery/Redis explicitly marked "future" in the stack — avoids premature infra for a single-developer build
PostgreSQL in production, SQLite in dev	PostgreSQL everywhere	Zero-friction local development while keeping identical SQLAlchemy models
Polling for live metrics (v1.0)	WebSocket streaming	Simpler to implement and debug; WebSockets revisited only if polling cadence proves insufficient
Redact-only storage for Security Intelligence findings	Store raw matched secret for user review	Eliminates an entire class of secondary data-leak risk from within Infralytix itself

8 Security & Scalability Design

How NFRs from the SRS become concrete design

Security Design

- JWT access tokens (short-lived) + refresh tokens; validated in a FastAPI dependency shared by every protected route
- RBAC enforced at the service layer, not just the route layer, so a Viewer role cannot mutate data even via a crafted request
- GitHub OAuth tokens stored encrypted at rest; never forwarded to the frontend
- Security Intelligence findings persist only a redacted fingerprint + file/line reference, per the data dictionary in Section 4

Scalability Design

- FastAPI instances are stateless — horizontal scaling behind a load balancer requires no sticky sessions
- Each Agent is a pure function over (project context, request) → response, making it straightforward to move to async workers later without a redesign
- Heavy analysis jobs (large repo scans) are structured to be queue-ready: the synchronous v1.0 call signature matches what a future Celery task signature would need

NEXT DOCUMENT IN THIS SUITE

Document 4 — Test Plan & Test Cases: the testing strategy and concrete test cases that verify every requirement and design element above.